

03

Lecture 03

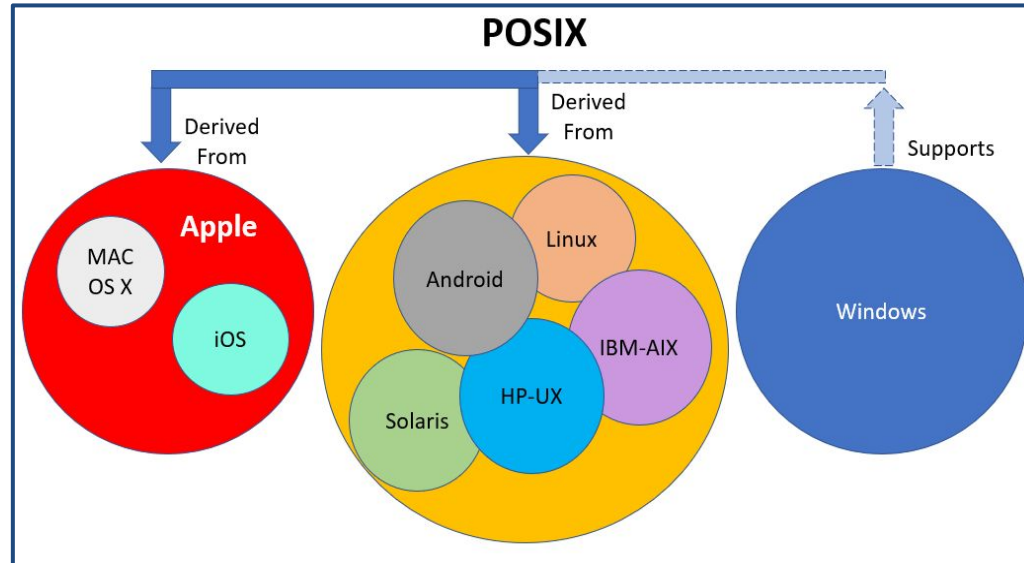
Process Creation, Termination & Threads

by - Brijesh Shukla
C07: Operating Systems

Join the lecture online on your dashboard.

POSIX (Portable Operating System Interface):

It is a set of standards (**rulebook**) created to **make different operating systems behave in a similar**, predictable way — especially Linux/ Unix/ macOS/ Solaris - like systems.



POSIX (Portable Operating System Interface) :

Apple (macOS & iOS): macOS (formerly Mac OS X) and iOS are UNIX-certified and POSIX compliant (that means Apple's operating systems follow POSIX rules directly).

- macOS uses the XNU kernel (hybrid kernel created by Apple), which has both:
- Mach (microkernel ideas)
 - BSD (UNIX-based, POSIX compatible)

Note: Before POSIX, every UNIX version was different. Programs written for one system often did not run on another.

So POSIX standardizes:

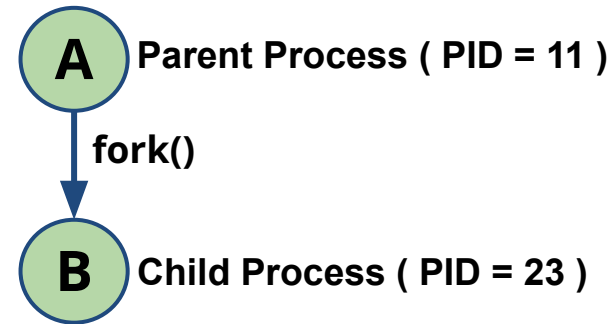
- fork()
- exec()
- open(), read(), write()
- Threads (pthreads)
- File permissions
- Shell commands

fork() System Call :

fork(): it is a POSIX system call used to create a new process. Basically, it creates a copy of the current running program.

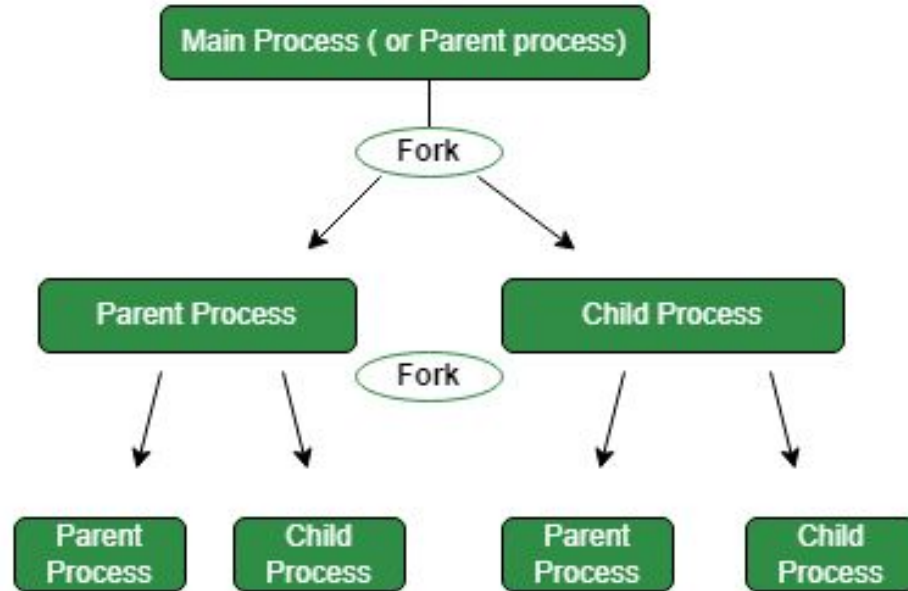
The child is an almost exact copy of the parent:

- same code
- same memory layout: Code, Data, Stack (but separate memory!)
- same environment
- same open file descriptors
- Allocates new PID for the child
- Child gets copy of parent's address space, PCB



Note: Each process will continue executing independently. Both processes resume execution from the next line after fork()

fork() System Call :



Example - Analogy : Photocopy Machine

Imagine you (the parent process) give a page to a photocopy machine:

- The machine creates a duplicate page → this is the child process.
- Both pages have the same content.
- But now there are two independent pages.
- You can write on your page (parent)
- Someone else can write on the duplicate page (child)
- These do not affect each other



Parent Page

fork() System Call :

fork(): it is a POSIX system call used to create a new process. Basically, it creates a copy of the current running program.

The child is an almost exact copy of the parent:

- same code
- same memory layout: Code, Data, Stack (but separate memory)
- same environment
- same open file descriptors
- Allocates new PID for the child



Note: Each process will continue executing independently. Both processes resume execution from the next line after fork().

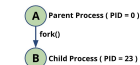
Child Page

fork() System Call :

fork(): it is a POSIX system call used to create a new process. Basically, it creates a copy of the current running program.

The child is an almost exact copy of the parent:

- same code
- same memory layout: Code, Data, Stack (but separate memory)
- same environment
- same open file descriptors
- Allocates new PID for the child



Note: Each process will continue executing independently. Both processes resume execution from the next line after fork().

fork() System Call :

```
#include <stdio.h>
#include <unistd.h>
int main(){
    pid_t id = fork();
    if (id == 0) {
        printf("This is Child Process, pid = %d\n",id);
    }
    else {
        printf("This is Parent Process. Child PID = %d\n", id);
    }
    return 0;
}
```

This is Parent Process. Child PID = 93334
This is Child Process, pid = 0

Output

This is Child Process, pid = 0
This is Parent Process. Child PID = 6549

Return Value	Process	Meaning
0	Child	You are inside the new child process
>0 (positive)	Parent	Value = Child's PID
-1	Parent	fork failed, no child created

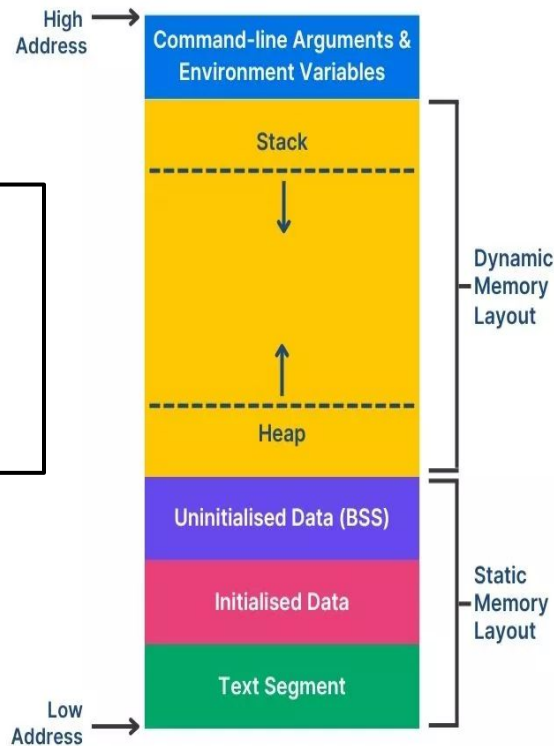
Memory Model After fork() : Copy-on-Write (COW) Optimization

When fork() is called:

- The OS does **not** immediately copy the entire memory.
- It uses **Copy-On-Write (COW)**:
 - Parent and child **share** memory pages at first.
 - If either modifies a page → OS creates a **separate copy**.

Note:

Text segment: shared (read-only)
Data, Heap, Stack: logically copied



Memory Model After fork() : Copy-on-Write (COW) Optimization

```
#include <stdio.h> // Required for printf
#include <unistd.h> // Required for fork() and sleep()
#include <sys/types.h> // Required for pid_t
#include <stdlib.h> // Required for pid_t exit()
int main() {
    int x = 10;
    // fork() returns pid_t (a type of integer)
    pid_t pid = fork();
    if (pid < 0) {
        // perror automatically appends the specific reason for failure
        perror("fork failed");
        exit(1); }
    else if (pid == 0) { // Child process
        x = 20;
        printf("Child x = %d\n", x);
    }
    else { // Parent process
        // Parent sleeps to allow child to potentially execute first
        sleep(1);
        printf("Parent x = %d\n", x);
    }
    return 0;
}
```

**This Kid Sleeps... But Why
Should Our Code?**



Output

Child x = 20
Parent x = 10

Memory Model After fork() : Copy-on-Write (COW) Optimization

BEFORE fork();

Parent: x = 10

AFTER fork (shared memory, COW):

Parent: x = 10 (shared page)

Child: x = 10 (shared page)

AFTER CHILD writes x = 20:

Parent: x = 10 (keeps original page)

Child: x = 20 (gets a new copied page)

AFTER fork (shared memory, COW):

Child x = 20

Parent x = 10

```
int x = 10;
// fork() returns pid_t (a type of integer)
pid_t pid = fork();
if (pid < 0) {
    // perror automatically appends the specific reason for failure
    perror("fork failed");
    exit(1);
}
else if (pid == 0) { // Child process
    x = 20;
    printf("Child x = %d\n", x);
}
else { // Parent process
    // Parent sleeps to allow child to potentially execute first
    sleep(1);
    printf("Parent x = %d\n", x);
}
```

Note:

- Parent & child share memory initially
- Pages marked read-only
- Copy occurs only when write happens
- Saves memory & time
- Most child processes call exec() immediately → no copying required

vfork() System Call :

vfork(): is like fork() but with NO copying or COW.

Child Shares:

- Parent's stack
- Parent's data segment
- Parent's heap
- Parent's globals
- Runs in parent's address space

Parent process is completely suspended

**Child must do only:
exec() or _exit()**

Note: macOS man pages describe vfork() as “obsolete”.

Dangerous because:

- Child can modify parent memory
- Stack clashing is possible
- Deprecated in modern systems

exec() System Call Family - Program Replacement :

- **exec()** is a system call that replaces the current running program with a new program.
- No new process is created — the same process ID continues.
- After a successful **exec()**, old code is gone and cannot resume

fork() → creates a new process (duplicate parent)

exec() → replaces process program with another (throw away old program & load new one)

Why exec() is needed? : When you run a program like - **./a.out**

Internally

1. Shell performs **fork()** → creates a child process
2. Child calls **exec("./a.out")**
3. Shell (parent) remains alive for further commands

exec() System Call Family - Program Replacement :

Examples of exec() functions:

execl(), execl(), execlp() - list arguments

execv(), execve(), execvp() - vector (array) arguments

execve() - is the actual system call, others are library wrappers

Function	Args	PATH	ENV	Notes
execl()	list	✗	inherited	Manual list
execlp()	list	✓	inherited	Most common "l"
execlp()	list	✗	✓	Custom env
execv()	vector	✗	inherited	Array
execvp()	vector	✓	inherited	Most used overall
execve()	vector	✗	✓	system call used by kernel

exec() System Call Family - Program Replacement :

Replaced by New Program	Remains Same
Code (Text)	Process ID (PID)
Data segment	File descriptors (unless close-on-exec)
Heap	User/group IDs
Stack	Some environment values

Syntax of exec() System Call: `exec(path, arg0, arg1, ..., NULL)`

Example:

```
exec("/bin/ls", "ls", "-l", "-a", NULL);
```

```
argv[0] = "ls"  
argv[1] = "-l"  
argv[2] = "-a"  
argv[3] = NULL
```

path — Where the executable is located
arg0 — Program name
arg1, arg2, arg3, ... — Command-line arguments
NULL — Marks end of arguments

exec1() System Call :

```
#include <unistd.h>
#include <stdio.h>
int main() {
    printf("Before exec()\n");
    exec1("/bin/ls", "ls", NULL);
    printf("This will NOT print if exec is successful\n");
}
```

NOTE: The second printf never runs — because exec() replaced the program.

Output

Before exec()		
b.out	fork1	sec.c
e2	fork1.c	tempCodeRunnerFile
e2.c	forkcomplete	tempCodeRunnerFile.c
exee.out	forkcomplete.c	vfork
exee1	forkcow	vfork.c
exee1.c	forkcow.c	ww
first	hello.txt	ww.c
first.c	sec	

exec() System Call :

```
exec("/bin/ls", "ls", "-l", "-a", NULL);
```

Output

```
Before exec()
total 1112
drwxr-xr-x@ 29 anitarani staff 928 Dec 2 06:13 .
drwx-----+ 39 anitarani staff 1248 Dec 2 05:31 ..
-rwxr-xr-x@ 1 anitarani staff 33426 Dec 1 10:45 b.out
-rwxr-xr-x@ 1 anitarani staff 33426 Dec 1 15:28 child
-rw-r--r--@ 1 anitarani staff 94 Dec 1 15:28 child.c
-rwxr-xr-x@ 1 anitarani staff 33455 Dec 2 06:13 e2
-rw-r--r--@ 1 anitarani staff 203 Dec 2 06:10 e2.c
-rwxr-xr-x@ 1 anitarani staff 33631 Dec 1 16:06 e3
-rw-r--r--@ 1 anitarani staff 878 Dec 1 15:30 e3.c
-rwxr-xr-x@ 1 anitarani staff 33429 Dec 1 10:44 exee.out
-rwxr-xr-x@ 1 anitarani staff 33426 Dec 1 10:22 exee1
fork1.c
-rwxr-xr-x@ 1 anitarani staff 33506 Dec 1 14:56 vfork
-rw-r--r--@ 1 anitarani staff 269 Dec 1 14:56 vfork.c
-rwxr-xr-x@ 1 anitarani staff 33503 Nov 24 06:56 ww
-rw-r--r--@ 1 anitarani staff 252 Nov 24 06:51 ww.c
(base) anitarani@ANITAs-MacBook-Air CPROGRAMS %
```

Examples of exec() functions :

```
execl("/bin/ls", "ls", "-l", "-a", NULL);
```

```
extern char **environ; // to use current environment  
execle("/bin/ls", "ls", "-l", "-a", NULL, environ);
```

```
execlp("ls", "ls", "-l", "-a", NULL);
```

```
char *args[] = {"ls", "-l", "-a", NULL};  
execv("/bin/ls", args);
```

```
extern char **environ;  
char *args[] = {"ls", "-l", "-a", NULL};  
execve("/bin/ls", args, environ);
```

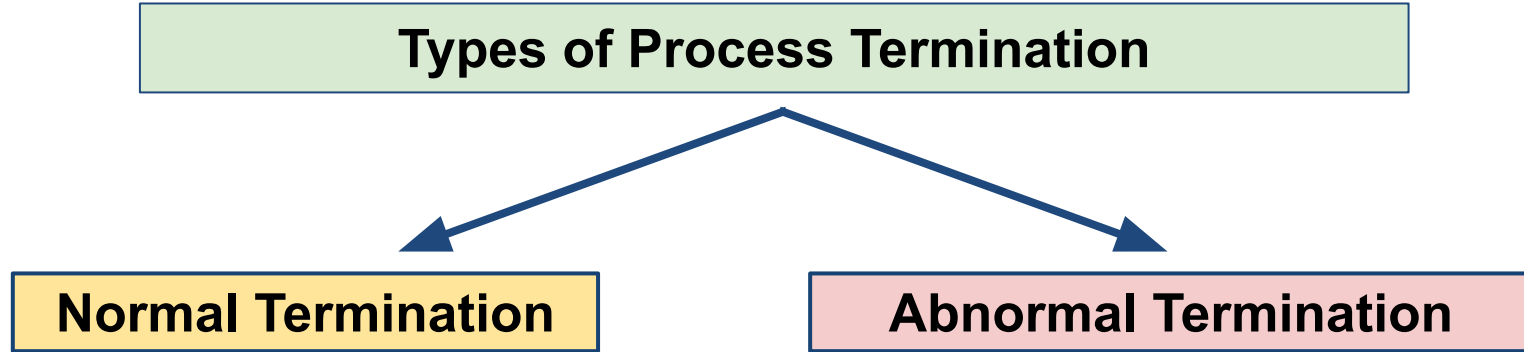
```
char *args[] = {"ls", "-l", "-a", NULL};  
execvp("ls", args);
```

execve() System Call :

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process - replace with ls command
        char *args[] = {"/bin/ls", "-l", "/tmp", NULL};
        char *env[] = {NULL};

        printf("Child about to exec...\n");
        execve("/bin/ls", args, env);

        // If we reach here, exec failed
        perror("execve failed");
        return 1;
    } else {
        // Parent waits for child
        printf("Parent waiting for child %d\n", pid);
        wait(NULL);
        printf("Parent: child finished\n");
    }
    return 0;
}
```



A process exits gracefully with proper cleanup and resource deallocation.

Characteristics:

- All open file descriptors are closed
- Buffers are flushed to disk
- Exit handlers (atexit) are called
- Exit status is returned to parent
- Resources are properly deallocated

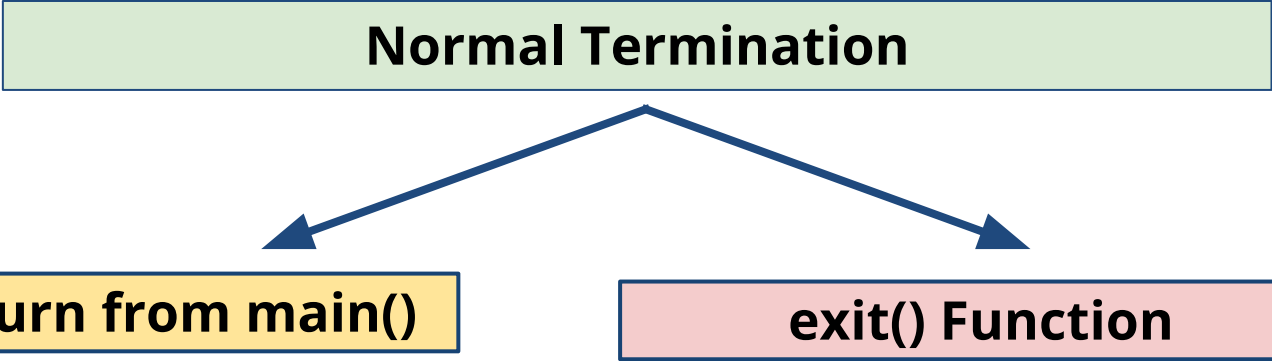
A process is terminated unexpectedly without proper cleanup.

Characteristics:

- Caused by signals or fatal errors
- No cleanup handlers executed
- Resources may not be released properly
- May leave corrupted or incomplete data
- Immediate termination

Normal Termination – Example

Normal Termination



```
graph TD; A[Normal Termination] --> B[return from main()]; A --> C[exit() Function];
```

return from main()

```
int main() {  
    // program logic return exit_status;  
    // 0 = success, non-zero = error }
```

exit() Function

Library function that performs cleanup before terminating the process.

Syntax

```
#include <stdlib.h>  
  
void exit(int status);
```

status: Exit code (0 = success, non-zero = error)

exit() :

What exit() Does (in order)

1. Calls all functions registered with atexit() in reverse order (LIFO)
2. Flushes all open stdio streams
3. Closes all open streams
4. Removes temporary files created by tmpfile()
5. Calls _exit() to enter kernel

Normal Termination - Example

```
#include <stdio.h>

int main() {
    printf("Processing data...\n");
    printf("Task completed successfully\n");
    return 0; // Normal exit with status 0
}
```

Note:

Normal termination means the process end by:

- Reaching **return 0** in main
- Calling **exit()**
- Finishing execution normally

Instead, the process was **forcibly killed by the operating system** due to an illegal operation.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1); // Normal termination with error code
    }
    else if (pid == 0) {
        // Child process
        printf("Child process executing\n");
        exit(0); // Normal termination with success
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: Child completed normally\n");
        exit(0);
    }
}
```

atexit() registration function:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void cleanup1() {
    printf("Cleanup handler 1 executed\n");
}
void cleanup2() {
    printf("Cleanup handler 2 executed\n");
}
void cleanup3() {
    printf("Cleanup handler 3 executed\n");
}
int main() {
}
```

/* Output:

Program started (PID: 84435)
Main program logic executing
Child process (PID: 84440)
Cleanup handler 3 executed
Cleanup handler 2 executed
Cleanup handler 1 executed
Parent: Child completed
Parent calling exit()
Cleanup handler 3 executed
Cleanup handler 2 executed
Cleanup handler 1 executed

```
int main() {
    printf("Program started (PID: %d)\n", getpid());
    // Register cleanup handlers
    atexit(cleanup1); // Registered first
    atexit(cleanup2); // Registered second
    atexit(cleanup3); // Registered third
    printf("Main program logic executing\n");
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child process (PID: %d)\n", getpid());
        sleep(1);
        exit(0); // Child calls exit() - NO parent's atexit handlers!
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: Child completed\n");
        printf("Parent calling exit()\n");
        exit(0); // Parent terminates here
    }
    printf("This line never executes\n");
}
```


_exit() System Call:

Direct system call that immediately terminates the process without any cleanup.

Syntax

```
#include <unistd.h>
void _exit(int status);
```

Parameters:

- **status:** Exit code passed to parent (only lower 8 bits used)

What _exit() Does

1. Closes all file descriptors
2. Releases memory
3. Notifies parent process
4. Directly enters kernel mode
5. **NO cleanup, NO buffer flushing, NO atexit handlers**

_exit() : Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void cleanup1() {
    printf("Cleanup handler 1 executed\n");
}
void cleanup2() {
    printf("Cleanup handler 2 executed\n");
}
void cleanup3() {
    printf("Cleanup handler 3 executed\n");
}
int main() {
    printf("Program started (PID: %d)\n", getpid());
    // Register cleanup handlers
    atexit(cleanup1); // Registered first
    atexit(cleanup2); // Registered second
    atexit(cleanup3); // Registered third
    printf("Main program logic executing\n");
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
    }
    else {
        // Parent process
    }
    printf("This line never executes\n");
}
```

```
int main() {
    .....
    if (pid == 0) {
        // Child process
        printf("Child process (PID: %d)\n", getpid());
        sleep(1);
        _exit(0); // Child calls exit() - NO parent's atexit handlers!
        /* Why _exit() instead of exit()?
        If we use exit() here:
        1. Would call cleanup() (parent's handler) - WRONG!
        2. Would flush parent's stdio buffers - causes duplicates
        3. Would close parent's file descriptors - corruption
        _exit() avoids all these problems!
        */
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: Child completed\n");
        printf("Parent calling exit()\n");
        exit(0); // Parent terminates here
    }
    printf("This line never executes\n");
}
```

_exit() : Example

/* Output:

Program started (PID: 84635)
Main program logic executing
Child process (PID: 84636)
Parent: Child completed
Parent calling exit()
Cleanup handler 3 executed
Cleanup handler 2 executed
Cleanup handler 1 executed

Abormal Termination :

Signal	Number	Description	Can Catch?
SIGKILL	9	Immediate termination	✗ No
SIGTERM	15	Graceful termination request	✓ Yes
SIGSEGV	11	Segmentation fault	✓ Yes
SIGABRT	6	Abort signal	✓ Yes
SIGINT	2	Interrupt (Ctrl+C)	✓ Yes
SIGFPE	8	Floating point exception	✓ Yes
SIGBUS	10	Bus error	✓ Yes
SIGQUIT	3	Quit (Ctrl+)	✓ Yes

Abnormal Termination - Example

```
#include <stdio.h>
int main() {
    int *ptr = NULL;
    printf("About to cause segmentation fault\n");
    *ptr = 10; // SIGSEGV - Abnormal termination
    return 0; // Never reached
}
```

Note:

Abnormal termination means the process didn't end by:

- Reaching `return 0` in main
- Calling `exit()`
- Finishing execution normally

Instead, the process was **forcibly killed by the operating system** due to an illegal operation.

Output:

```
Child running (PID: 5001)
Parent sending SIGKILL to child
Child terminated abnormally
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child - infinite loop
        printf("Child running (PID: %d)\n", getpid());
        while(1) {
            sleep(1);
        }
    }
    else {
        // Parent
        sleep(2);
        printf("Parent sending SIGKILL to child\n");
        kill(pid, SIGKILL); // Abnormal termination
        wait(NULL);
        printf("Child terminated abnormally\n");
    }
    return 0;
}
```

wait() System Call :

The wait() family of system calls allows a parent process to wait for its child processes to terminate and collect their exit status.

Why Do We Need wait()?

- **Synchronization:** Parent waits for child to complete
- **Zombie Prevention:** Reaps terminated children (cleans up PCB)
- **Status Collection:** Gets child's exit code
- **Resource Management:** Releases system resources

Syntax

```
pid_t wait(int *status);
```

status:

Pointer to integer where exit status is stored
(can be NULL if you don't care)

Return Value

- **Success:** PID of terminated child
- **Failure:** -1 (no children exist)

How wait() Works

1. Parent calls wait()
2. Parent **blocks** (sleeps) until ANY child terminates
3. Kernel wakes parent when a child exits
4. Returns child's PID and stores exit status

wait() : Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) { // Child process
        printf("Child: I'm working...\n");
        sleep(2);
        printf("Child: Done! Exiting with code 42\n");
        exit(42);
    } else { // Parent process
        printf("Parent: Waiting for child %d\n", pid);
        int status;
        pid_t child_pid = wait(&status);
        printf("Parent: Child %d terminated\n", child_pid);
    }
    return 0;
}
```

Output

Parent: Waiting for child 91078

Child: I'm working...

Child: Done! Exiting with code 42

Parent: Child 91078 terminated

waitpid() System Call :

Syntax

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Parameters

pid: Which child to wait for

- > 0: Wait for specific child with this PID
- 1: Wait for ANY child (same as wait())
- 0: Wait for any child in same process group
- < -1: Wait for children in process group |pid|

status: Pointer to store exit status

options: Control behavior (0 for blocking)

Return Value

- **Success:** PID of child that terminated
- **Failure:** -1

Zombie & Orphan Processes :

When working with `fork()` and process creation, two special situations can occur:

- **Zombie Process:** A process that has finished executing but still exists in the system
- **Orphan Process:** A process whose parent has died

Why Do These Exist?

These states arise from the parent-child relationship in Unix/Linux/macOS systems:

- Parent creates child with `fork()`
- Parent should wait for child with `wait()`
- But what if parent doesn't wait? → Zombie
- What if parent dies first? → Orphan

Zombie Processes :

Process Termination Flow

1. Child calls `exit(42)`
- ↓
2. Kernel cleans up child's resources
- ↓
3. Kernel keeps minimal PCB with exit status
- ↓
4. Child becomes ZOMBIE (waiting to be reaped)
- ↓
5. Parent calls `wait()`
- ↓
6. Parent receives exit status (42)
- ↓
7. Kernel removes PCB - zombie is gone

Zombie State Characteristics

Property	Status
Executing?	✗ No
Has memory?	✗ No
Has open files?	✗ No
Has PCB?	✓ Yes (minimal)
Visible in <code>ps</code> ?	✓ Yes (as <code><defunct></code> or <code>Z</code>)

Note: Calling `wait()` or `waitpid()` removes the zombie and cleans up the PCB completely.

Zombie Processes : Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    printf("Parent PID: %d\n", getpid());
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) { // Child process
        printf("Child PID: %d\n", getpid());
        printf("Child: I'm done, exiting...\n");
        exit(0); // Child exits immediately
    }
    // Parent process
    printf("Parent: NOT calling wait()\n");
    printf("Parent: Child %d is now a ZOMBIE!\n", pid);
    printf("Parent: Run 'ps aux | grep Z' to see it\n");
    sleep(30); // Parent sleeps so zombie stays visible
    printf("Parent: Exiting (zombie will be cleaned by init)\n");
    return 0;
}
```

Output 1

Parent PID: 1234
Child PID: 1235
Child: I'm done, exiting...
Parent: NOT calling wait()
Parent: Child 1235 is now a ZOMBIE!
Parent: Run 'ps aux | grep Z' to see it

Output 2

Parent PID: 74830
Parent: NOT calling wait()
Parent: Child 74834 is now a ZOMBIE!
Parent: Run 'ps aux | grep Z' to see it
Child PID: 74834
Child: I'm done, exiting...
Parent: Exiting (zombie will be cleaned by
init)

Zombie Processes : Example

How to handle Zombie Process ?



Use: wait(NULL)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    printf("Parent PID: %d\n", getpid());
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) { // Child process
        printf("Child PID: %d\n", getpid());
        printf("Child: I'm done, exiting...\n");
        exit(0); // Child exits immediately
    }
    // Parent process , ADD wait(NULL) HERE:
    wait(NULL);
    printf("Parent: I have reaped the child. It is NO LONGER a zombie.\n");
    printf("Parent: Now I can sleep safely.\n");
    sleep(30);
    printf("Parent: Exiting.\n");
    return 0;
}
```

Orphan Processes :

Process Termination Flow

1. Parent (PID 100) creates Child (PID 200)
Parent: PPID=1, Child: PPID=100
2. Parent (PID 100) exits
3. Kernel detects orphan (Child PID 200)
4. Kernel changes Child's PPID to 1
Child: PPID=1 (now child of init/launchd)
5. Child exits
6. launchd/init automatically calls wait()
7. No zombie created!

Orphan State Characteristics

Property	Status
Executing?	✓ Yes (still running)
Has resources?	✓ Yes (normal process)
Has parent?	✓ Yes (adopted by init/launchd)
Is this a problem?	✗ No (system handles it)

Orphan Processes : Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {
        // Child process
        printf("Child: My original parent PID: %d\n", getppid());
        printf("Child: Sleeping to become an orphan...\n");
        sleep(5); // Give parent time to exit
        // After parent exits, the child is adopted by init (or systemd/launchd)
        printf("\nChild: My new parent PID (after orphan): %d\n", getppid());
        printf("Child: Exiting now.\n");
        exit(0);
    }
    else { // Parent process
        sleep(1); // so that child start first
        printf("Parent: I am exiting now, leaving my child alone...\n");
        exit(0); // Parent exits immediately
    }
    return 0;
}
```

Output

Child: My original parent PID: 84557

Child: Sleeping to become an orphan...

Parent: I am exiting now, leaving my child alone...

Child: My new parent PID (after orphan): 1

Child: Exiting now.

Orphan Processes : Example

How to handle Orphan Process ?



Use: wait(NULL)

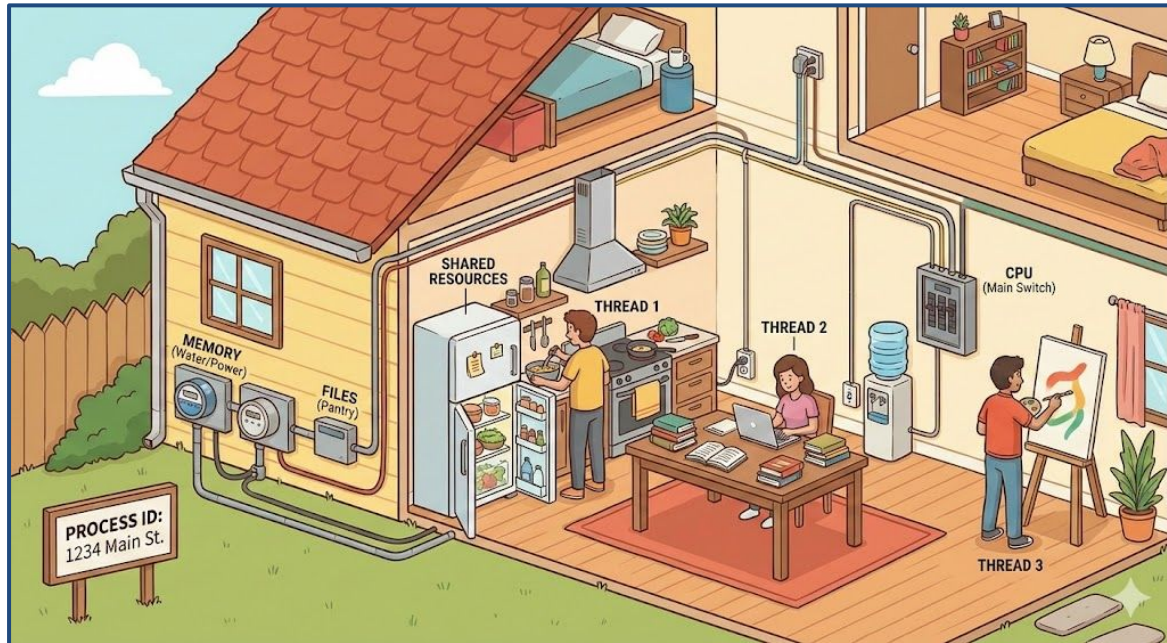
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h> // Required for wait()
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) { // Child process
        printf("Child: My parent PID: %d\n", getppid());
        printf("Child: Working for 5 seconds...\n");
        printf("Child: Finished work. Exiting...\n");
        sleep(5);
        exit(0);
    }
    else { // Parent process
        printf("Parent: Waiting for my child (PID %d) to finish...\n", pid);
        // SOLUTION: The parent blocks here and waits for the child
        int status;
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child finished normally with exit code %d.\n", WEXITSTATUS(status));
        }
        printf("Parent: Child is reaped. Parent is now exiting safely.\n");
        exit(0);
    }
    return 0;
}
```

Thread: The Lightweight Unit of Execution

Real-World Analogy:

Process = A house with its own address, utilities, and furniture

Thread = Multiple people living in the same house, sharing resources but doing different tasks



Thread: The Lightweight Unit of Execution

A thread is a lightweight unit of execution within a process that shares process resources but executes independently.

Also known as: "Lightweight Process"

Aspect	Process	Thread
Address Space	independent, separate	Shared within process
Resources	Separate copy	Shared
Context Switch	Heavy	Light (same memory space)
Creation Cost	High	Low
Communication	IPC mechanisms needed	Direct shared memory

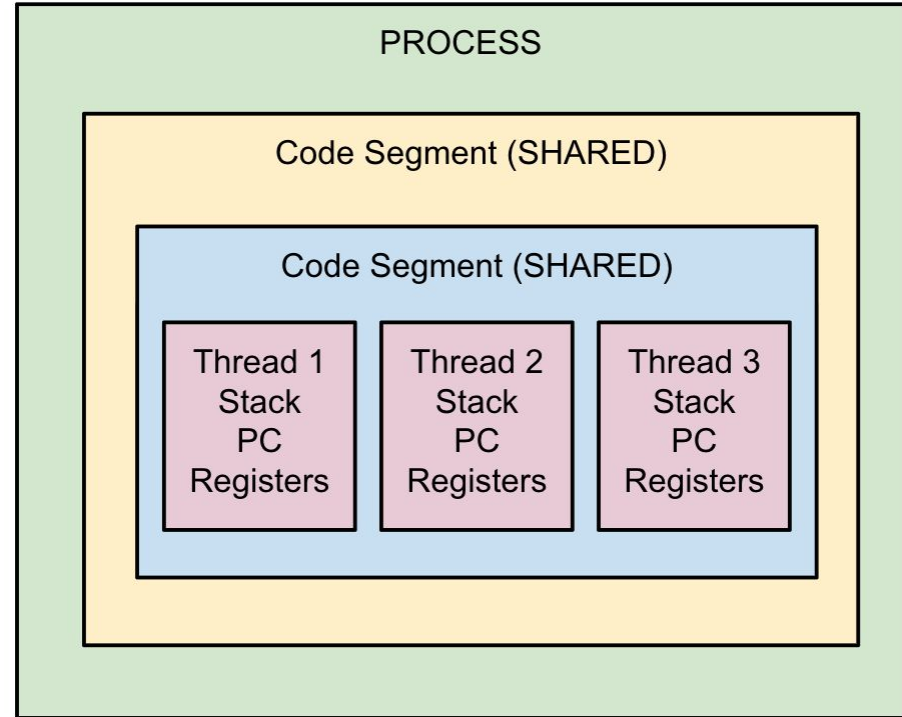
Thread Memory Model - What's Shared vs Separate

What Threads **SHARE** (Within Same Process):

-  Code segment (program instructions)
-  Data segment (global variables)
-  Heap (dynamically allocated memory)
-  Open files and file descriptors
-  Signal handlers
-  Working directory
-  Process ID (PID)
-  User and group IDs

What Each Thread Has **SEPARATELY**:

-  Thread ID (TID)
-  Program Counter (PC) - tracks execution position
-  Register set - CPU registers
-  Stack - local variables, function calls
-  Signal mask
-  Scheduling priority



Benefits of Multithreading:

- 1 Performance
- 2 Easy Communication
- 3 Responsiveness
- 4 Resource Utilization
- 5 Modularity

Real-World Example: Web Browser

Thread 1: Renders UI

Thread 2: Downloads images

Thread 3: Executes JavaScript

Thread 4: Handles user input

All sharing browser's memory, running concurrently!



Thank You!